<JS_Asíncrono/>

Dominando la Asincronía: Desde el Event Loop hasta Async/Await

Guía de estudio progresiva y práctica para desarrolladores

NotebookLM

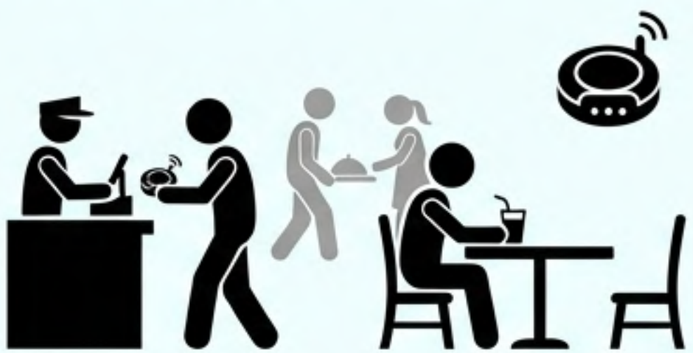
Síncrono (Bloqueante)

- Una tarea a la vez.
- El código se detiene hasta terminar.
- **Metáfora:** Esperar en la fila del restaurante. Nadie más puede pedir hasta que te entreguen tu comida.

```
while(true) { esperar(); }
```

Asíncrono (No Bloqueante)

- Múltiples tareas inician sin detener el programa.
- **Metáfora:** Pides tu comida, recibes un localizador y te sientas. El localizador avisa cuando está listo.

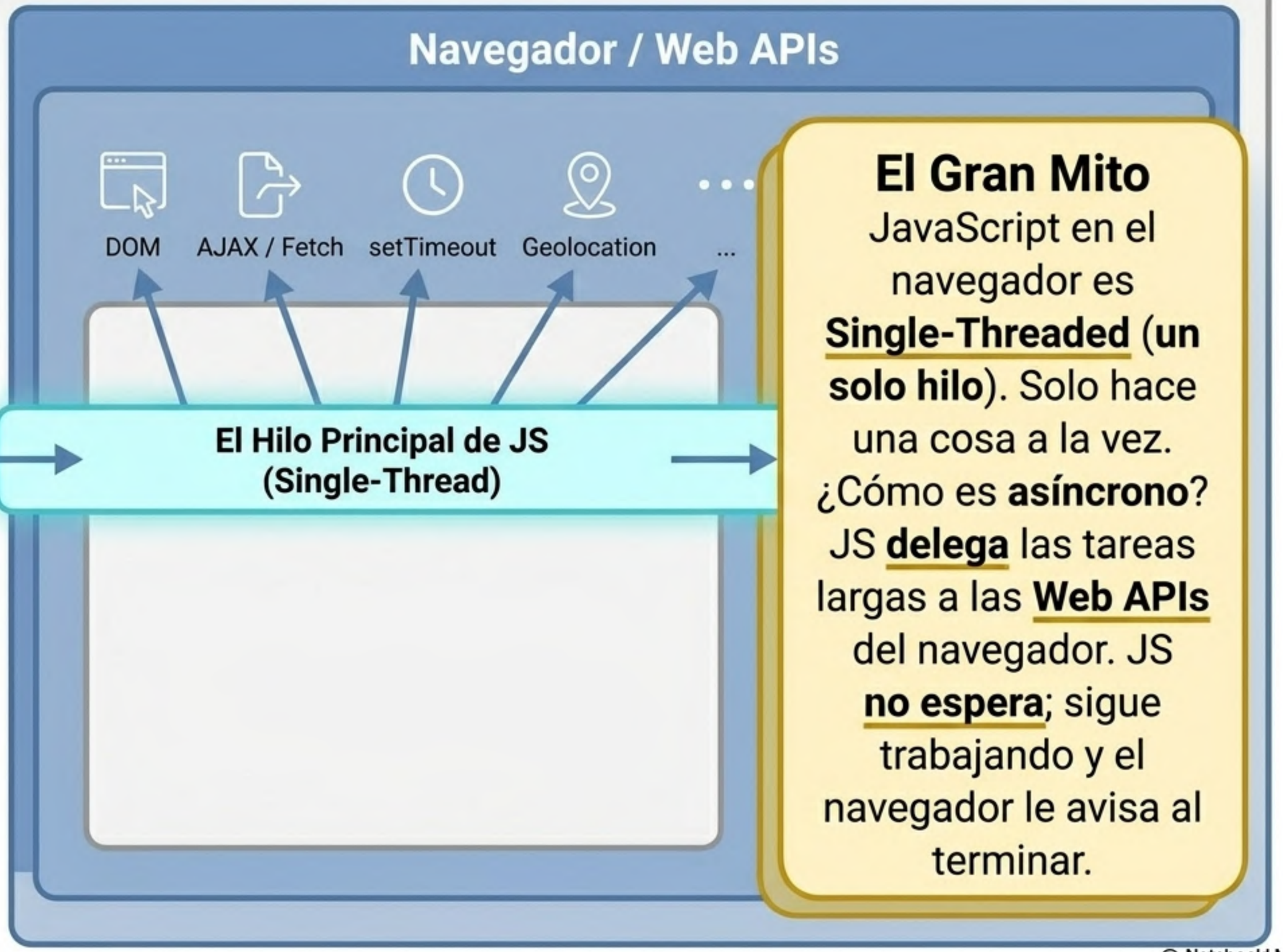


```
setTimeout(() => comer(), 1000);
```

NotebookLM

Multithreading vs Single-Thread

- **Thread (Hilo):** La unidad básica de ejecución.
- **Concurrencia**: Lidiar con muchas tareas.
- **Paralelismo**: Ejecutar tareas exactamente al mismo tiempo.



El Gran Mito

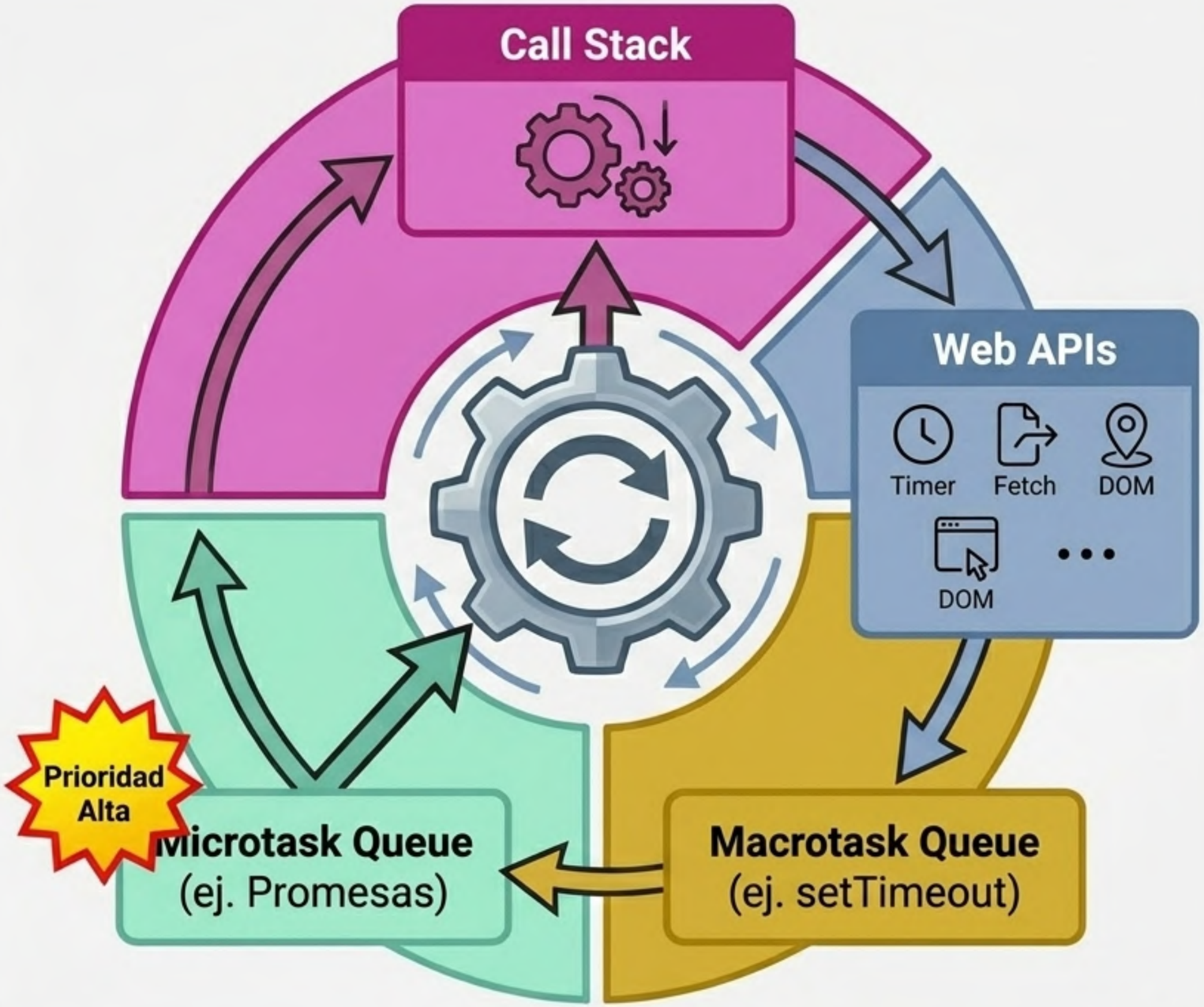
JavaScript en el navegador es **Single-Threaded** (**un solo hilo**). Solo hace una cosa a la vez. ¿Cómo es **asíncrono**? JS **delega** las tareas largas a las **Web APIs** del navegador. JS **no espera**; sigue trabajando y el navegador le avisa al terminar.

NotebookLM

El Motor de Todo: The Event Loop

1. El motor ejecuta código en el Call Stack hasta vaciarlo.
2. Las tareas asíncronas esperan en las Web APIs.
3. Al terminar, los callbacks pasan a las colas (Queues).
4. El Event Loop revisa: Si el Stack está vacío, empuja tareas de las colas al Stack.

Regla de Oro: ¡Las **Microtasks** siempre tienen **prioridad absoluta** sobre las **Macrotasks**!



NotebookLM

Definición

Una función que se pasa como argumento a otra función para ser ejecutada después de que una operación asíncrona termine.

Metáfora

Como dejarle tu número a un técnico: "Haz tu trabajo y, cuando termines, llámame a este número".

El Código

```
console.log('1');
setTimeout(() => {
  console.log('2 (Callback)');
}, 0);
console.log('3');
```

Análisis

El resultado en consola será: 1, 3, 2.

Un setTimeout con 0 ms no significa ejecución inmediata, significa "ejecutar en cuanto el Call Stack esté libre".

NotebookLM

El Problema: Callback Hell

- Conocido como "La Pirámide de la Perdición".
- Ocurre al realizar múltiples acciones asíncronas en secuencia, dependiendo cada paso del anterior.
- Impacto: El código crece hacia la derecha, no hacia abajo. La legibilidad colapsa y manejar errores requiere lógica repetitiva en cada nivel.

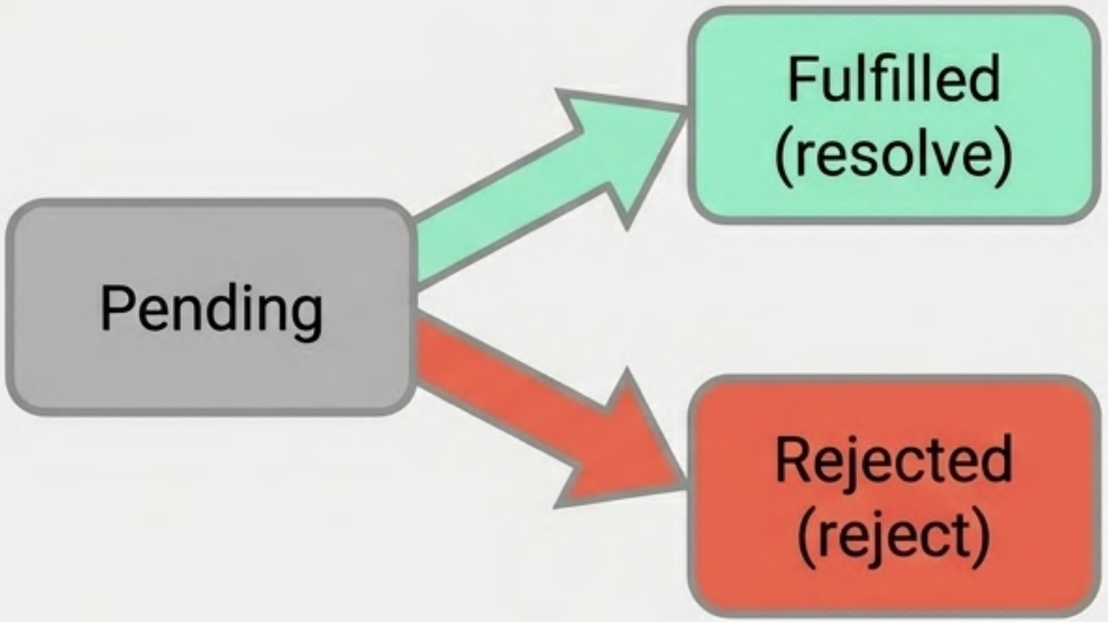
```
cargarScript('1.js', (error, script) => {
  if (error) manejarError(error);
  cargarScript('2.js', (error, script) => {
    if (error) manejarError(error);
      cargarScript('3.js', (error, script) => {
        // Código ilegible y difícil de mantener
      });
  });
});
```

NotebookLM

Promesas

Un objeto que representa la terminación o el fracaso eventual de una operación asíncrona.

Metáfora: El cantante y los fans. La Promesa es la lista de suscripción. Si la canción sale, o el estudio se incendia, todos los fans son notificados.




```
const miPromesa = new Promise((resolve, reject) => {
  // Código asíncrono aquí
  if (exito) resolve('¡Canción lista!');
  else reject(new Error('¡Fuego en el estudio!'));
});
```

NotebookLM

Consumiendo Promesas

Una promesa por sí sola no hace nada útil sin sus suscriptores. Usamos métodos para reaccionar a los cambios de estado.

- **.then()**: Se ejecuta cuando la promesa es Fulfilled. Recibe el resultado.
- **.catch()**: Se ejecuta cuando la promesa es Rejected. Atrapa el error.
- **.finally()**: Se ejecuta siempre al terminar, sin importar el resultado. Ideal para limpieza.

```
miPromesa
  .then(resultado =>
console.log(resultado))
  .catch(error =>
console.error(error.message))
  .finally(() =>
console.log('Operación
terminada'));
```

Ruta de Éxito

Ruta de Error

NotebookLM

Encadenamiento (Chaining)

- Cada vez que llamamos a **.then()**, este devuelve una nueva promesa.
- Permite encadenar acciones secuenciales de forma plana (el código crece hacia abajo).
- Magia del Manejo de Errores: Si cualquier eslabón falla, la ejecución salta automáticamente al **.catch()** final. ¡Un solo manejador para múltiples operaciones!

```
cargarScript('1.js')
.then(script1 => cargarScript('2.js'))
.then(script2 => cargarScript('3.js'))
.then(script3 => iniciarApp())
.catch(error => console.error('Falló la carga'));
```

NotebookLM

La Revolución: Async / Await

Azúcar sintáctico para escribir código asíncrono que se lee como síncrono.

Antes: Promesas

- Cadenas de .then()
- Callbacks internos

```
function inicializar() {
  cargarScript('1.js')
    .then(s1 => cargarScript('2.js'))
    .then(s2 => cargarScript('3.js'))
    .then(s3 => iniciarApp());
}
```

Ahora: Async / Await

- **async**: Obliga a la función a devolver Promesa.
- **await**: Pausa la ejecución sin bloquear el hilo.

```
async function inicializar() {
  const s1 = await cargarScript('1.js');
  const s2 = await cargarScript('2.js');
  const s3 = await cargarScript('3.js');
  iniciarApp();
}
```

NotebookLM

Manejo de Errores con Async/Await

- **El Problema:** Al eliminar los .then(), también eliminamos la estructura donde vivía el .catch().
- **La Solución:** Usamos el bloque clásico try...catch.
- **El Súper Poder:** Atrapa errores síncronos y asíncronos en el mismo lugar, unificando toda la lógica de manejo de errores de tu aplicación.

```
async function obtenerDatos() {
  try {
    const respuesta = await fetch('/api/datos');
    const datos = await respuesta.json();
    return datos;
  } catch (error) {
    console.log('¡Atrapado!', error.message);
  }
}
```

NotebookLM

Ejecución Paralela: Promise.all

Úsalo cuando tienes múltiples tareas asíncronas que no dependen unas de otras y quieres que corran en paralelo para ganar tiempo.

```
const resultados = await Promise.all([
  fetch('/usuarios'),
  fetch('/productos'),
  fetch('/configuracion')
]);
```

Peligro Oculto

`Promise.all` se rechaza inmediatamente si una sola promesa falla. Sin embargo, las demás promesas no se cancelan, siguen ejecutándose en segundo plano y pueden causar errores silenciosos.

NotebookLM

Comparación Final: Cuándo usar qué

	Callbacks	**Promesas (.then)**	**Async / Await**
Legibilidad Secuencial	❌ Pobre (Callback Hell)	✅ Buena (Cadenas planas)	✅ Excelente (Parece síncrono)
Manejo de Errores	❌ Difícil (Repetitivo)	✅ Centralizado (.catch)	✅ Unificado (try...catch)
Ejecución Paralela	❌ Muy Complejo	✅ Nativo (Promise.all)	✅ Nativo (Promise.all + await)
Veredicto final / Cuándo Usar	Eventos DOM o setTimeout	Flujos paralelos masivos	Estándar moderno para el 95% del código

NotebookLM